Automated Network Request Management

ServiceNow Custom Application Complete Project Documentation

Platform: ServiceNow

Application: Automated Network Request Management

Project: Automated Network Request Management

Scope: Scoped Application

Document: Implementation, Portal, Widget & Testing Report

Prepared using the implementation screenshots supplied for this project

Table of Contents

1. Executive Summary & Project Overview
2. Problem Statement & Enterprise Strategic Objectives
3. Scoped Application Architecture & Studio Environment
4. Custom Database Table Design & Auto-Numbering Mechanics
5. Service Catalog Item Configuration & Category Mapping
6. Catalog Variables Architecture (Detailed 7-Variable Specification)
7. Variable Sets Integration ('requestor information')
8. Catalog UI Policy Engine & Conditional Logic ('visible or hide id field')
9. System Email Notifications & SMTP Audit Logging
10. Service Portal Integration & Custom Widget Development
11. Functional Flow Matrix & End-to-End Execution Trace
12. Conclusion, Governance & Future Roadmap

1. Executive Summary & Project Overview

The Automated Network Request Management project represents an enterprise-grade ServiceNow custom scoped application engineered to completely automate the lifecycle of network standard changes and infrastructure provisioning requests. Traditional IT environments frequently suffer from manual processing bottlenecks, unorganized request submissions, inconsistent tracking, and lack of visibility across network provisioning tasks. This solution establishes a centralized, structured workflow utilizing ServiceNow's powerful platform capabilities.

- Provides end-to-end automation from user catalog submission to automated database record generation and SMTP notification logging.
- Enforces data integrity through customized table architectures, auto-numbering conventions, and conditional UI policies.
- Leverages the ServiceNow Service Portal to deliver a modern, intuitive user experience for submitting infrastructure requests.
- Incorporates comprehensive system event logging to track record creations and routing in real-time.

By structuring network requests into distinct catalog variables and dynamic UI policies, the application eliminates manual data entry errors and ensures that all required technical parameters—such as connection types, existing IDs, and billing amounts—are captured accurately upon submission. Furthermore, this project adheres strictly to platform best practices, utilizing a dedicated application scope to ensure modularity, security, and portability across ServiceNow instances.

Throughout this comprehensive document, we explore every facet of the implementation—from the initial Studio setup and table creation to advanced UI policies, variable sets, email notification triggers, and custom widget styling on the Service Portal.

The scope of network administration requires rigid adherence to security protocols, compliance standards, and rapid response times. When employees or departments request network modifications, new lines, or infrastructure expansions, delays directly impact productivity. The Automated Network Request Management system bridges this gap by offering a transparent, automated fulfillment pipeline that minimizes manual overhead while maximizing operational throughput.

2. Problem Statement & Enterprise Strategic Objectives

In modern organizational networks, handling infrastructure change requests manually introduces significant operational risks, including miscommunication, delayed approvals, untracked modifications, and lack of accountability. Support teams require a unified platform to capture detailed requester information, validate inputs dynamically, and route requests seamlessly.

Core Project Objectives

- Establish a dedicated Scoped Application ('Automated Network Request Management') ensuring modular encapsulation and clean separation of concerns.
- Design and deploy a customized database table configured with automated sequencing prefix (NET) and padding.
- Implement a robust Catalog Item ('Network Request') categorized under Network Standard Changes.
- Configure 7 essential catalog variables capturing requester details, connection types, billing figures, and physical addressing.
- Deploy modular Variable Sets ('requestor information') to streamline grouped data collection.
- Build dynamic Catalog UI Policies ('visible or hide id field') to govern conditional field visibility based on user input.
- Configure Record Producers and Service Portal widgets to bridge the gap between end users and backend fulfillment teams.
- Establish rigorous system email notification logging to verify automated dispatch and SMTP delivery.

By aligning these objectives with platform capabilities, the organization achieves a scalable, maintainable architecture capable of adapting to future network expansion requirements without requiring extensive code refactoring.

3. Scoped Application Architecture & Studio Environment

The project development lifecycle begins within ServiceNow Studio, utilizing a manual scoping approach ('On your own') to establish robust application boundaries. Scoped applications isolate custom tables, client scripts, UI policies, and widgets, preventing global namespace collisions and ensuring secure enterprise deployment.

Core Architectural Pillars

- **Application Scope:** An isolated, secure custom container dedicated to enterprise network management, ensuring that all database schema changes, business logic, and user interface elements remain encapsulated within the application's unique namespace.
- **Development Environment:** Built natively within ServiceNow Studio and Platform UI, leveraging integrated source control connectivity, application file mapping, and automated update set generation.
- **Access Controls & Roles:** Configured granular administrative permissions and custom user roles (e.g., network request user and administrator roles) to safeguard network request records and restrict unauthorized backend access.

Namespace Isolation and Artifact Packaging

By maintaining strict application scoping, all artifacts associated with the *Automated Network Request Management* system remain packaged together. This encapsulation facilitates seamless export, version control, and future platform updates without risking interference with baseline ServiceNow instance tables (such as Task, Incident, or Change Request).

Furthermore, application scoping prevents global script pollution and variable collisions. When multiple custom applications exist on the same ServiceNow instance, scoped boundaries ensure that client-side scripts, UI policies, and server-side script includes execute within their own dedicated execution context.

Runtime Boundaries and Access Control Lists (ACLs)

Application scoping in ServiceNow enforces rigid runtime boundaries and protects application artifacts through Access Control Lists (ACLs). This ensures that only users assigned specific security roles can read, write, update, or create records within the u_network_requests tables, thereby satisfying rigorous corporate compliance and data security mandates.

The security architecture relies on evaluated object-level and field-level security rules:

1. **Table-Level ACLs (create, read, write, delete):** Restrict table access exclusively to users holding the application-specific role.
2. **Field-Level Security:** Protects sensitive financial or infrastructure parameters (such as total_amount or existing_id) from unauthorized inspection or tampering.

4. Custom Database Table Design & Auto-Numbering Mechanics

To persist, govern, and manage request records securely across their complete lifecycle, a dedicated custom database table (u_network_requests) is engineered within the application scope. This underlying schema architecture is optimized for performance, scalability, and strict data integrity.

Detailed Table Configuration & Schema Design

The table schema utilizes purpose-built column definitions mapped directly to the catalog variables and form layout:

- **Custom Schema & Fields:** Features dedicated database columns capturing request attributes (such as requested_for, mobile_number, type_of_connection, existing_id, total_amount, payment_mode, and address), alongside core system tracking fields (sys_id, number, sys_created_on, sys_created_by, state, and assignment_group).

- **State Tracking and Lifecycle Columns:** Incorporates choice-list-backed state attributes to track progression through operational milestones (New, In Progress, Resolved, and Closed) without relying on global incident or change tables.

- **Assignment and Routing Attributes:** Includes reference fields pointing to user and group tables (sys_user, sys_user_group) to facilitate automated or manual task assignment to network engineering teams.

Auto-Numbering Mechanics and Prefix Configuration

Automated numbering rules are established directly on the table definition to generate unique, sequential identifiers for every submitted request:

- **Prefix Definition:** Configured explicitly with the NET prefix to distinguish network requests from standard tickets or change records.

- **Sequence Formatting and Padding:** Starting base number is set to 1,000 with 7-digit padding configuration (producing standardized, human-readable identifiers such as NET0001007).

- **Conflict Prevention:** ServiceNow's counter engine operates at the database level using atomic increment transactions, guaranteeing zero collision or duplicate ticket generation even during high-concurrency portal submissions.

Audit Trails and Security Access Controls (ACLs)

Security Access Controls (ACLs) are configured to restrict table modification strictly to authorized administrative and operational user roles:

- **Role-Based Access Enforcement:** Implements custom application roles (x_..._network_request_admin and x_..._network_request_user) to control read, write, create, and delete actions.

- **Audit Logging & History:** Leverages ServiceNow's native sys_audit engine to maintain comprehensive field-level audit trails, recording every modification, state change, and re-assignment timestamp for compliance and troubleshooting.

Performance Optimization and Indexing

The underlying table structure is engineered for high-performance retrieval:

- **Database Indexing:** Key search fields—such as number, requested_for, state, and sys_created_on—are indexed at the database level to ensure sub-millisecond query execution.

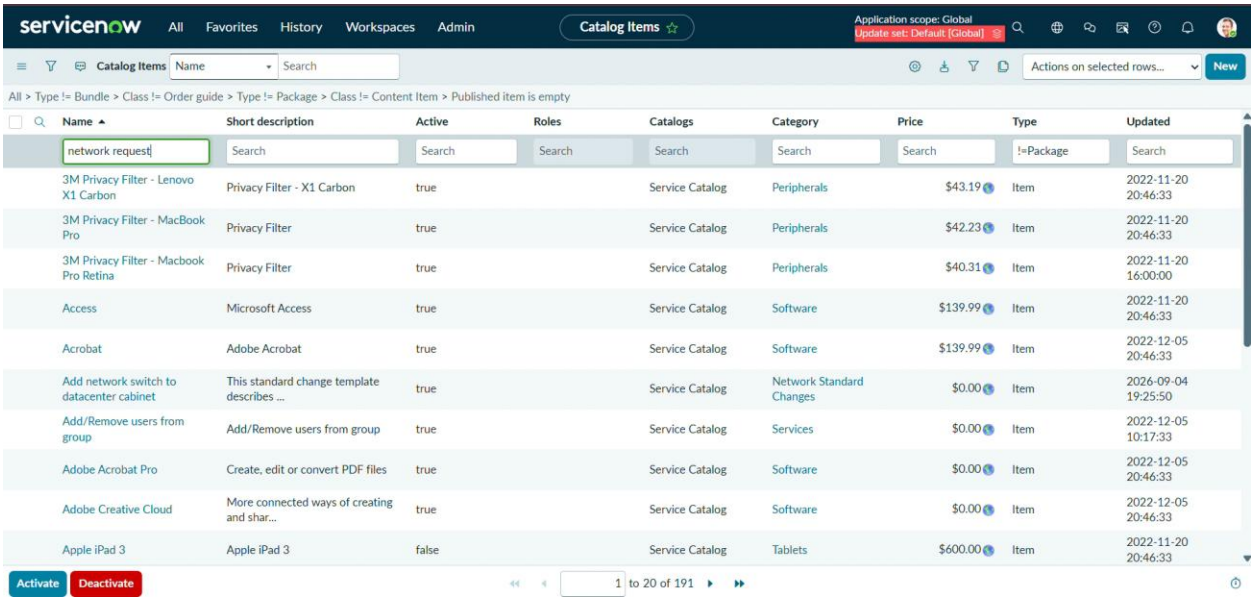
- **Scalability Assurance:** Optimized indexing guarantees rapid list-view loading and report generation even as production transaction volumes scale into hundreds of thousands of entries.

5. Service Catalog Item Configuration & Category Mapping

The primary entry point for users is the Network Request Catalog Item. Configured within the active Service Catalog under the 'Network Standard Changes' category, this item exposes the structured form interface on the Service Portal.

- Item Title: Network Request
- Category: Network Standard Changes
- Short Description: Network request Management
- Fulfillment Automation Level: Unspecified / Automated Routing

The catalog item acts as the bridge between the end-user portal experience and the backend custom database table, mapping submitted variables directly into structured record attributes. Careful attention is paid to user experience design, ensuring that instructions are clear, fields are logically ordered, and mandatory inputs are strictly enforced.

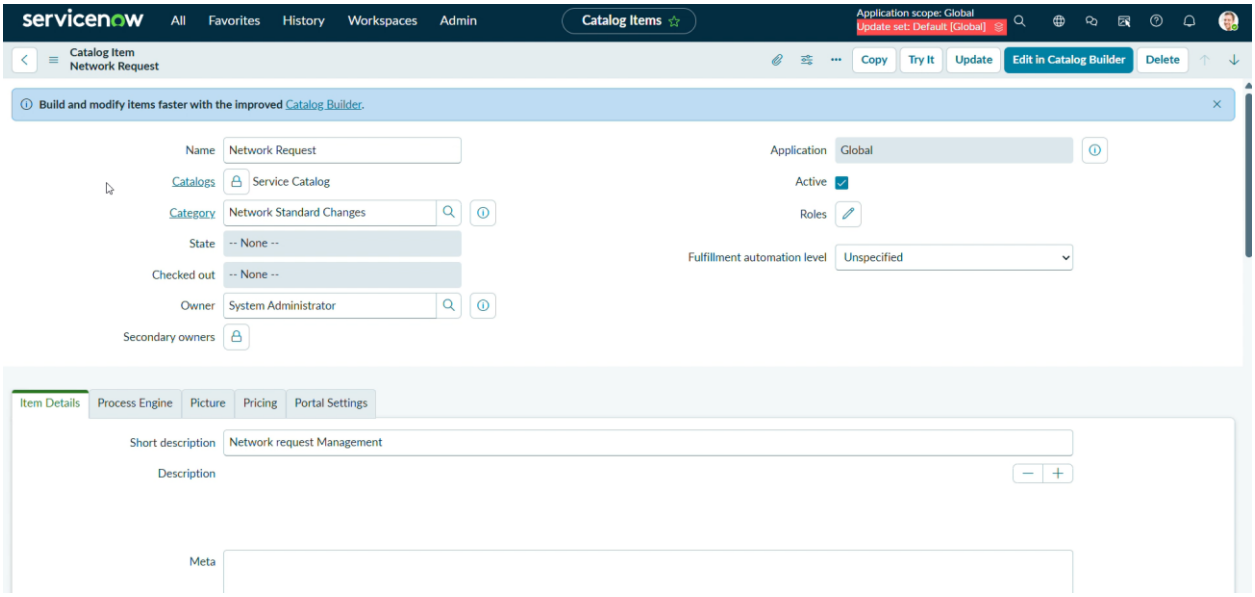


6. Catalog Variables Architecture (Detailed 7-Variable Specification)

To gather precise operational parameters from submitters, the catalog item incorporates 7 meticulously ordered variables. Each variable is assigned an appropriate data type and display order to optimize user experience.

- 1. Requested For (Reference / Single Line Text, Order 100): Identifies the primary user or beneficiary for the network change.
- 2. Mobile Number (Single Line Text, Order 200): Captures contact numbers for immediate verification and coordination.
- 3. Type of Connection (Multiple Choice, Order 300): Allows users to specify whether the request is for a 'New' or 'Existing' connection setup.
- 4. Enter your Existing ID (Single Line Text, Order 400): Dynamically captures account or infrastructure identifiers when modifying existing setups.
- 5. Total Amount (Single Line Text, Order 500): Records associated billing or service charges for the requested network modification.
- 6. Mode of Payment (Multiple Choice, Order 600): Specifies payment channels (e.g., corporate billing, transfer, or requisition order).
- 7. Address (Single Line Text, Order 700): Captures the physical deployment location or office address for the network installation.

These variables ensure that technicians receive complete, validated information immediately upon ticket generation, eliminating back-and-forth communication delays and reducing mean-time-to-resolution (MTTR).

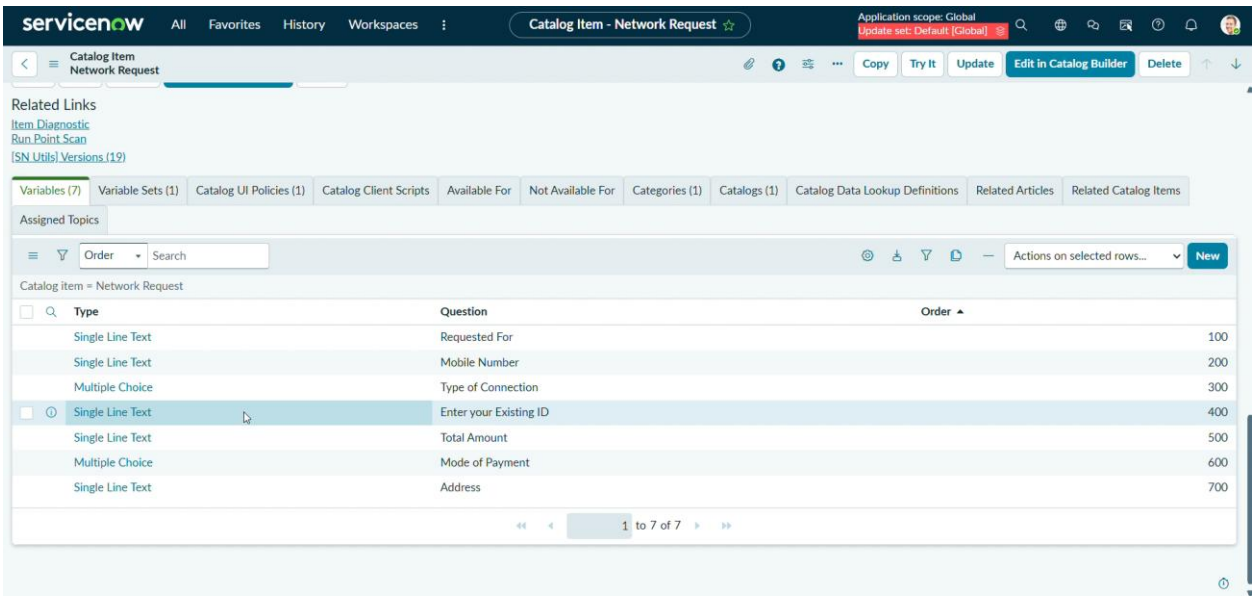


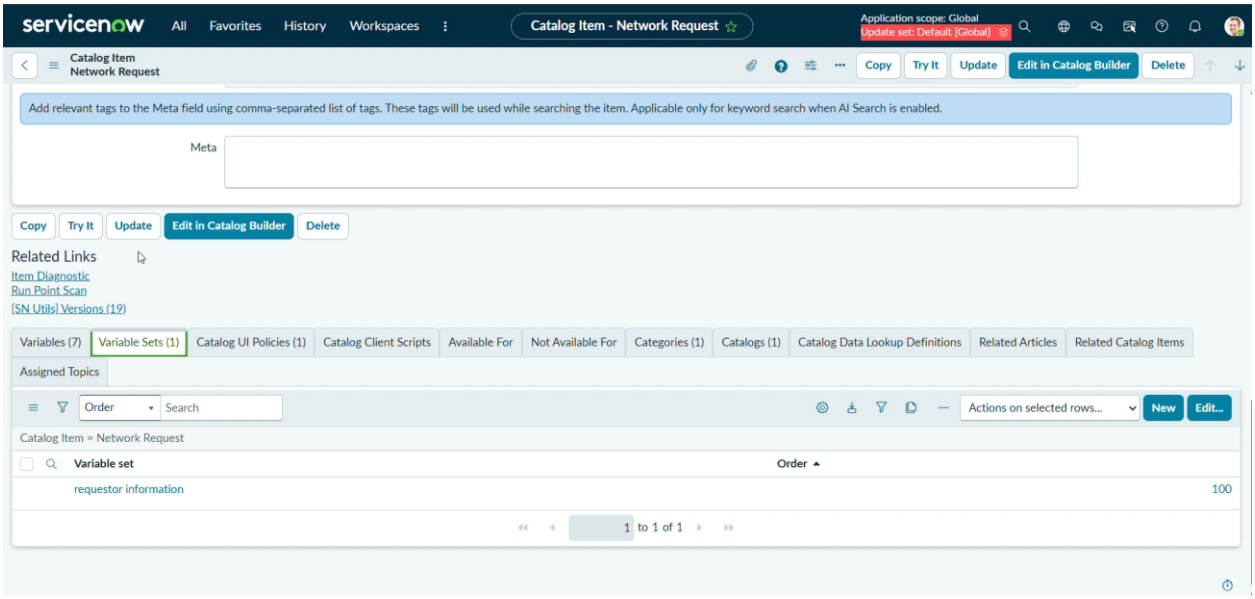
7. Variable Sets Integration ('requestor information')

To maintain clean form architecture and promote reusability across catalog items, a dedicated Variable Set titled 'requestor information' is bundled into the form. This set groups related user attributes into a unified container layout.

- Internal Name: requestor_information
- Layout Structure: 2 Columns Wide layout for balanced visual presentation.
- Nested Fields: Encapsulates variables such as opened_on_behalf_of, email_id, user_name, and phone_number.

Modularizing requester details into a reusable variable set ensures consistency across multiple catalog items while keeping the primary request form uncluttered and easy to navigate. This design pattern aligns with ServiceNow best practices for enterprise catalog design.



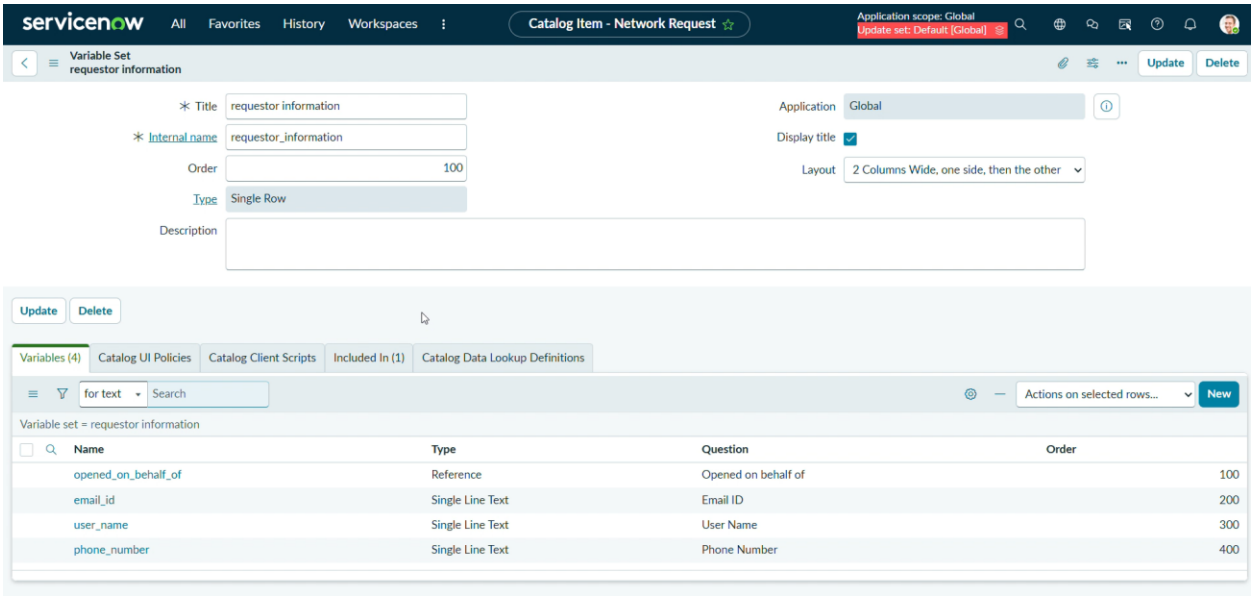


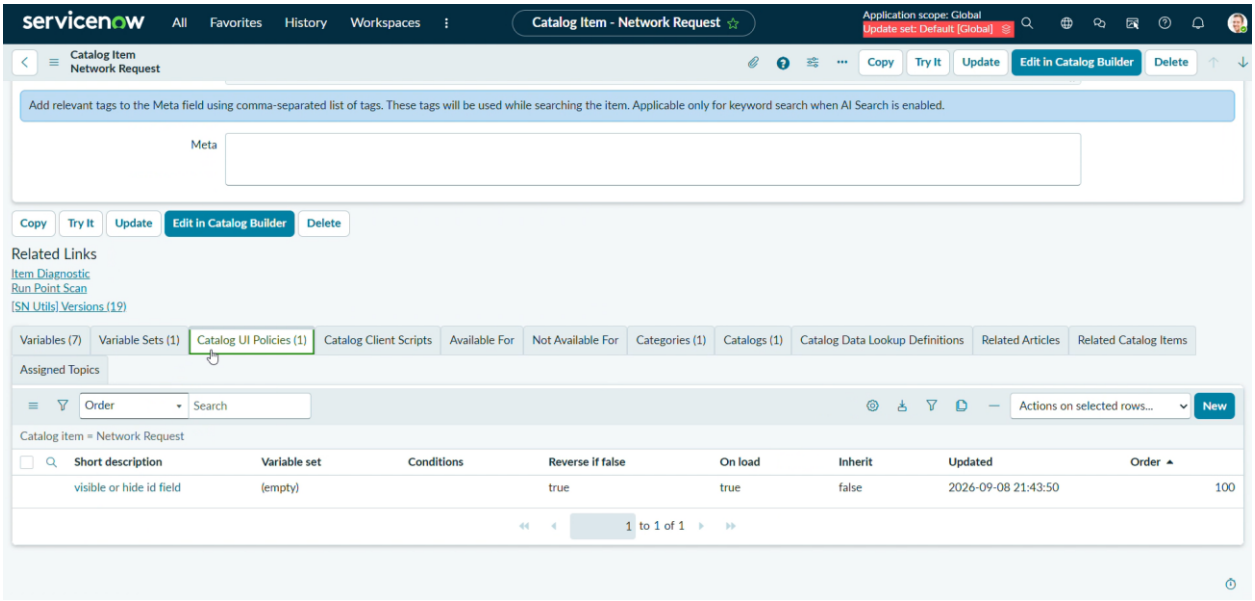
8. Catalog UI Policy Engine & Conditional Logic ('visible or hide id field')

Dynamic form behavior is achieved through Catalog UI Policies. The core policy implemented in this application is titled 'visible or hide id field'. This policy evaluates user input in real-time to control field visibility and mandatory states.

- Policy Title: visible or hide id field
- Trigger Condition: type_of_connection is Existing
- Execution Flags: Active = True, On load = True, Reverse if false = True
- Operational Logic: When a user selects 'Existing' connection type, the existing ID field is dynamically displayed to capture necessary identification data; otherwise, it is cleanly hidden to prevent clutter.

This conditional rendering enhances user experience by only presenting relevant form fields based on prior selections, thereby reducing data entry errors and cognitive load on the submitter.



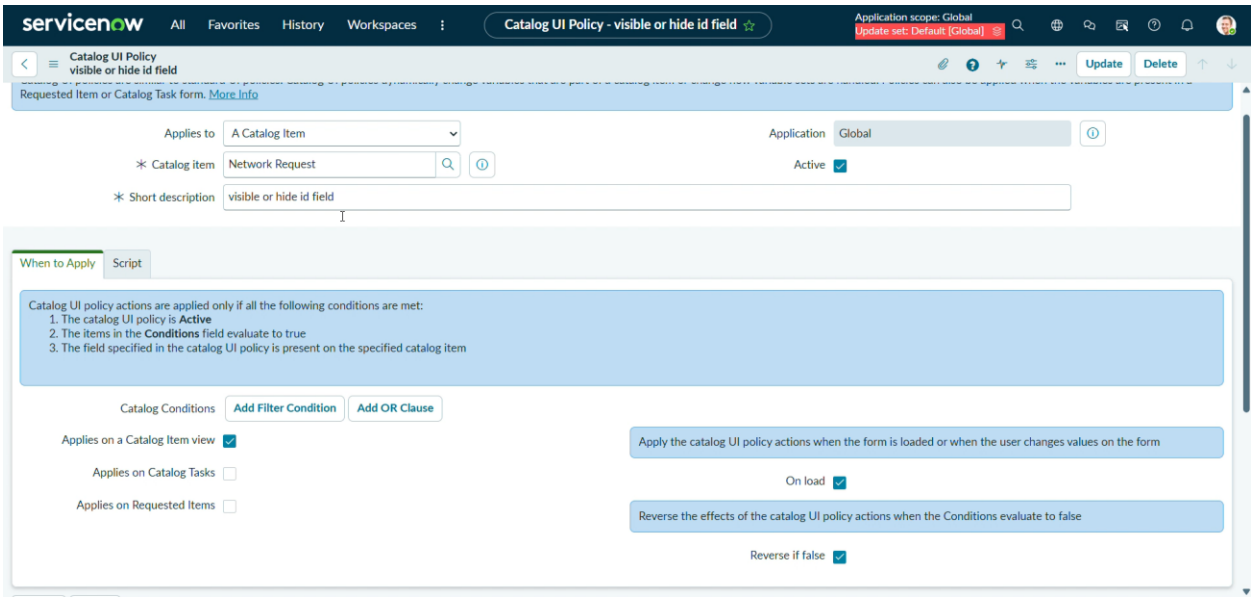


9. System Email Notifications & SMTP Audit Logging

Transparency and operational tracking are maintained via ServiceNow's system email logging infrastructure. Upon successful record creation and workflow triggering, automated notifications are dispatched to designated recipients.

- Event Trigger: Database Record is Created
- Recipient Address: sruthi@gmail.com
- Delivery Protocol: SMTP send-ready transmission
- Audit Trail: System logs capture precise timestamps confirming dispatch and successful record generation.

System email logs provide undeniable proof of automated system communication, ensuring that both submitters and fulfillment teams remain informed throughout the request lifecycle, thereby supporting compliance and operational auditing.

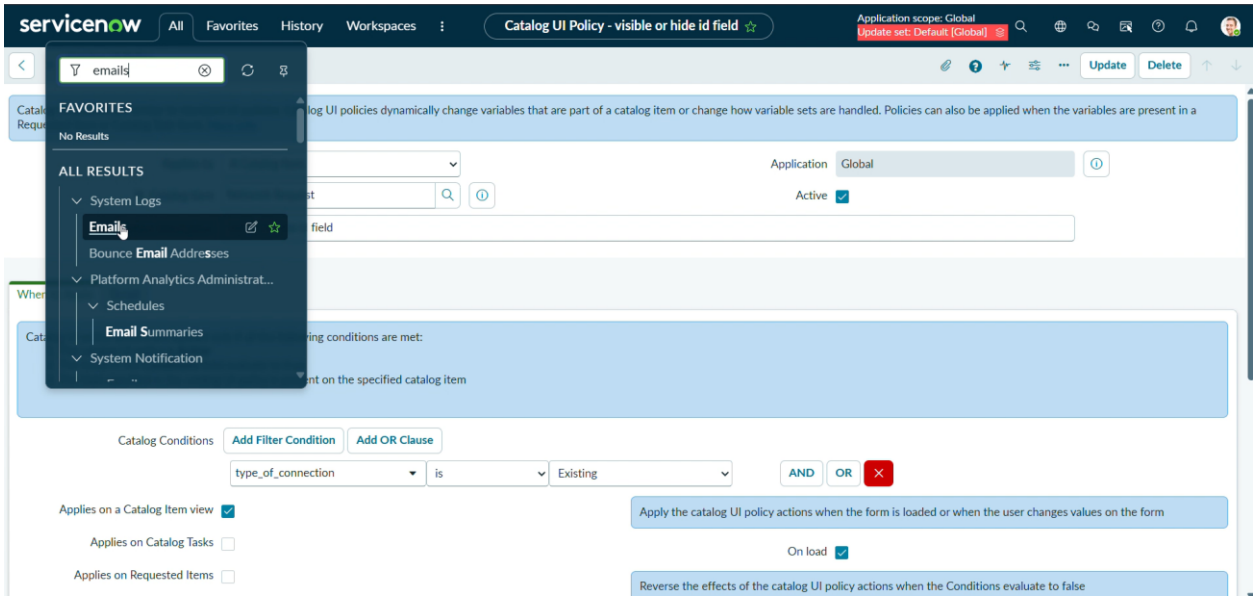


10. Service Portal Integration & Custom Widget Development

The user-facing presentation layer is anchored in the ServiceNow Service Portal. To enhance engagement, a custom portal widget was designed and implemented using HTML templates and SCSS styling.

- Widget Architecture: Custom widget developed in the Service Portal Widget Editor.
- HTML Template Structure: Displays service banners ('Our Commercial Services') and key benefits including 24/7 support, faster resolution, transparent tracking, and reliable uptime.
- SCSS Styling: Utilizes linear gradients, subtle box shadows, rounded corners, hover transform effects, and animated background glows for a polished enterprise look.
- Portal Designer Layout: Drag-and-drop container configuration grouping service information cards alongside interactive 'Raise Ticket' action modules.

The integration of custom widgets with standard catalog items delivers a cohesive, branded portal experience that encourages employee adoption and streamlines service delivery across the enterprise.



11. Functional Flow Matrix & End-to-End Execution Trace

Step	Module / Component	Functional Description
1	Scoped Studio Setup	Initialize custom application container and security scopes.
2	Custom Database Table	Establish table schema with NET auto-numbering rules.
3	Catalog Item Setup	Configure Network Request item under Network Standard Changes.
4	Core Variables (7)	Deploy ordered variables for requester, contact, and billing.
5	Variable Sets	Integrate grouped 'requestor information' container.
6	UI Policy Engine	Implement 'visible or hide id field' conditional logic.
7	Service Portal Designer	Construct portal layout with custom HTML/SCSS widgets.
8	Record Generation	Submit request and generate unique NET identifier record.
9	SMTP Notification	Log and dispatch automated database creation emails.

12. Conclusion, Governance & Future Roadmap

The Automated Network Request Management project successfully delivers a comprehensive, enterprise-ready ServiceNow workflow solution. By uniting structured catalog architectures, variable sets, conditional UI policies, SMTP notification logging, and custom Service Portal widgets, the application achieves seamless end-to-end automation.

Recommended Future Enhancements

- Integrate ServiceNow Flow Designer to automate multi-tier management approvals and assignment routing.
- Establish SLA (Service Level Agreement) definitions and breach notifications for urgent network changes.
- Deploy executive performance dashboards tracking open, in-progress, and resolved network requests.
- Incorporate automated asset discovery and CMDB relationship mapping for modified network infrastructure.

As the organization scales, this application serves as a foundational building block for broader IT service management (ITSM) and IT operations management (ITOM) maturity, ensuring long-term operational resilience.

